

VLSI Project Report

Guru Shwadeep Dornadula

2023112020

Guru.dornadula@research.iiit.ac.in

International Institute of Information Technology - Hyderabad

Abstract—This paper details the design and implementation of a 4-bit carry-lookahead adder (CLA) using 180nm CMOS technology. The adder, optimized for minimal delay, employs static CMOS logic and has been validated through NGSPICE simulations. The core components include propagate and generate blocks, a carry-lookahead circuit, and an output sum buffer, with post-layout verification performed in MAGIC to ensure timing compliance. Additionally, the design was implemented on an FPGA to verify functionality, with results confirming stable operation at the intended clock frequency.

I. INTRODUCTION

High-speed adders are critical components in digital systems, with the carry-lookahead adder (CLA) being a popular choice due to its efficient carry propagation mechanism. This project focuses on the implementation of a 4-bit CLA using 180nm CMOS technology, with an emphasis on minimizing delay. Various VLSI logic styles, including static CMOS, dynamic, and mixed logic, were evaluated to identify the most effective design approach. The CLA comprises propagate and generate logic for precomputing carry signals, a lookahead circuit for parallel carry generation, and a sum block with buffering to meet drive strength requirements. Simulations were conducted in NGSPICE to ensure proper timing and functionality, followed by layout and post-layout verification in MAGIC to analyze parasitic effects. The design was also implemented on an FPGA, validating reliable performance at high clock speeds. This comprehensive methodology highlights the CLA's ability to achieve low delay while meeting timing requirements for high-performance digital systems.

II. D-FLIPFLOP

Proposed Structure: To design an efficient D flip-flop, Complementary Pass-Transistor Logic (CPL) is employed. CPL utilizes PMOS and NMOS transistors as switches controlled by the clock signal. Compared to traditional CMOS logic, CPL circuits offer reduced power consumption and faster operation, making them well-suited for high-speed applications.

The latch design in CPL consists of interconnected NMOS and PMOS transistors with their drains and sources linked, followed by a static CMOS inverter to restore voltage levels. This configuration ensures proper signal integrity and cascading for building the D flip-flop. The proposed structure is designed with a total of 10 transistors, including 5 PMOS and 5 NMOS transistors.

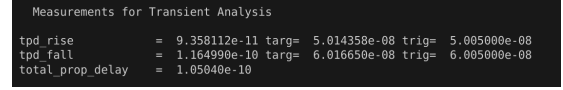


Fig. 1. D flip flop timing output.

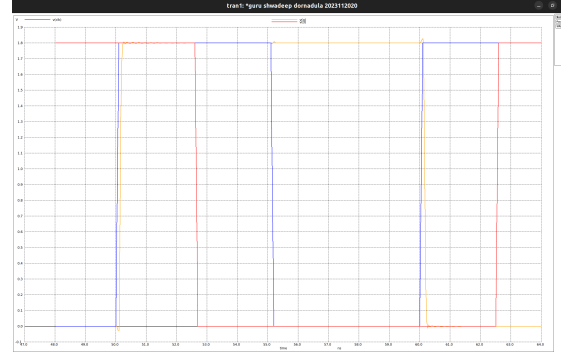


Fig. 2. D flip flop output.

A. ngspice

1) *Clock-to-Q delay*: The clock-to-Q delay of the flip-flop is evaluated using the NGSpice commands shown in fig. To achieve the least delay, PMOS and NMOS transistor dimensions are optimized to balance rise and fall times. The chosen dimensions are:

$$W_p = 24\lambda = 2.16 \mu\text{m}, \quad W_n = 10\lambda = 0.9 \mu\text{m}.$$

2) *set up and hold time*: Setup and hold times are determined by analyzing signal alignment with clock transitions:

setup time= 18ps

hold time = 63 ps

B. MAGIC layout

the layout of the D flip flop implemented in magic is as follows:

Post layout simulations give the following delays!

setup time= 20ps

hold time = 90ps

III. CLA ADDER

A. Proposed Structure

A regular adder cascades a bunch of 2 bit full adders. This causes a massive delay in the carry of a particular stage because it is cascaded through many gates. To avoid this we can pre-compute the carry's as follows and implement it

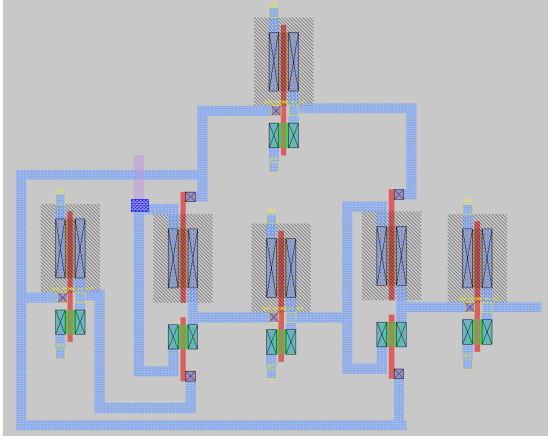


Fig. 3. D flip flop in magic.

directly. This will greatly improve performance as the number of bits we want to add increases. We generate two key signals: propagate (P_i) and generate (G_i) for each bit, which help in reducing the overall delay:

$$P_i = a_i \oplus b_i$$

$$G_i = a_i \cdot b_i$$

These signals allow the carry outputs to be calculated independently of the carry input from the previous stage, thus eliminating the sequential delay inherent in a cascade of full adders. This inherent parallelism is what makes the CLA faster than traditional ripple-carry adders.

$$c_1 = g_0 + p_0 c_{in} \quad (1)$$

$$c_2 = g_1 + p_1 c_1 = g_1 + p_1 g_0 + p_1 p_0 c_{in} \quad (2)$$

$$c_3 = g_2 + p_2 c_2 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_{in} \quad (3)$$

$$c_4 = g_3 + p_3 c_3 \quad (4)$$

$$c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_{in} \quad (5)$$

finnaly,

$$S_i = P_i \oplus C_{i-1}$$

In my implementation, I have used **static CMOS logic** for all the individual logic gates. I have made a 2-input OR-gate, a 2 input AND-gate, and 2 input XOR. In place of a N-input gate, i have cascaded the 2-input gates to achieve required functionality. I have done this because this makes the design and testing phase simpler.

B. Xor gate

this gate consists of 4Pmos and 4Nmos where

$$W_p = 48\lambda m, \quad W_n = 20\lambda m.$$

C. And gate

this gate consists of 3Pmos and 3Nmos. Its basically a NAND gate cascaded with a not gate. Sizing is:

$$W_p = 24\lambda m, \quad W_n = 20\lambda m.$$

D. Or gate

this gate consists of 3Pmos and 3Nmos. Its basically a NOR gate cascaded with a not gate. Sizing is:

$$W_p = 24\lambda m, \quad W_n = 20\lambda m.$$

E. Simulation Results

Upon simulating the proposed adder in NGSpice for various input combinations of A and B , the typical output sequence observed is as follows: the output carry (C_{out}) is generated first, followed by S_1 , S_2 , S_3 , and finally S_4 . This behavior is attributed to the relative complexity of the carry propagation path. The output carry often traverses a greater number of transistors compared to the intermediate carry bits, resulting in a delay.

As a consequence, the generation of S_4 is delayed since the sum bits (S_i) depend on their respective carry bits. This sequential dependency is consistent with the design and logic flow of the circuit.

It can therefore be concluded that the worst-case delay of the adder is determined by the S_4 bit. This delay is significantly higher than other configurations because it depends on the cumulative propagation of carry bits throughout the circuit. The sequential dependency of the sum bits on carry bits contributes to this elevated delay for S_4 .

The specific scenario leading to the maximum delay can be intuitively identified. The worst-case delay occurs when C_4 is generated through the path involving the maximum number of transistors. In this design, this path corresponds to the inputs $A = 1001$ and $B = 1111$.

F. magic layout

The overall design of the adder is divided into four parts, with each part responsible for producing the respective carry output. These carry outputs are subsequently passed through xor-gates to generate the sum bits. This circuit occupies a total area of The dimensions of the layout are $1140\lambda \times 754\lambda$ ($102.6 \times 67.86 = 6962.436^2$).

G. MAGIC Layout

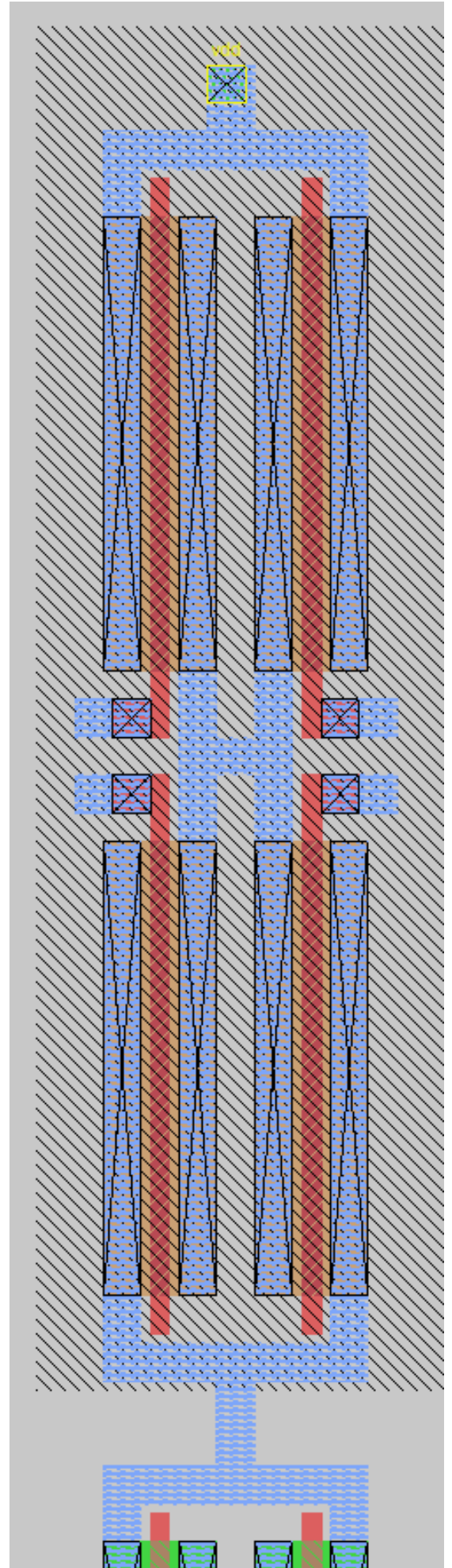
For the layout of the addition module, the inputs are provided to the D flip-flops, and the outputs are obtained after the D flip-flops. This circuit occupies a total area of $1792\lambda \times 1345\lambda$.

From static timing analysis, the minimum clock period is given by:

$$T_{clk} \geq T_{pd} + T_{pcq} + T_{su}$$

IV. FPGA

The Verilog HDL code was further synthesized and implemented on an FPGA using Vivado. Inputs $A = 1010$ and $B = 1110$ were fed to the FPGA via input pins. The corresponding output waveforms were observed on an oscilloscope, as shown in Figs.



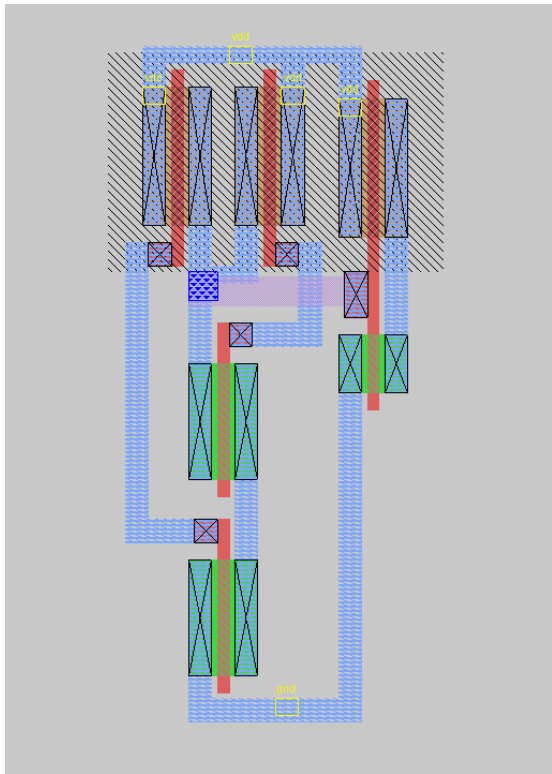


Fig. 5. and magic

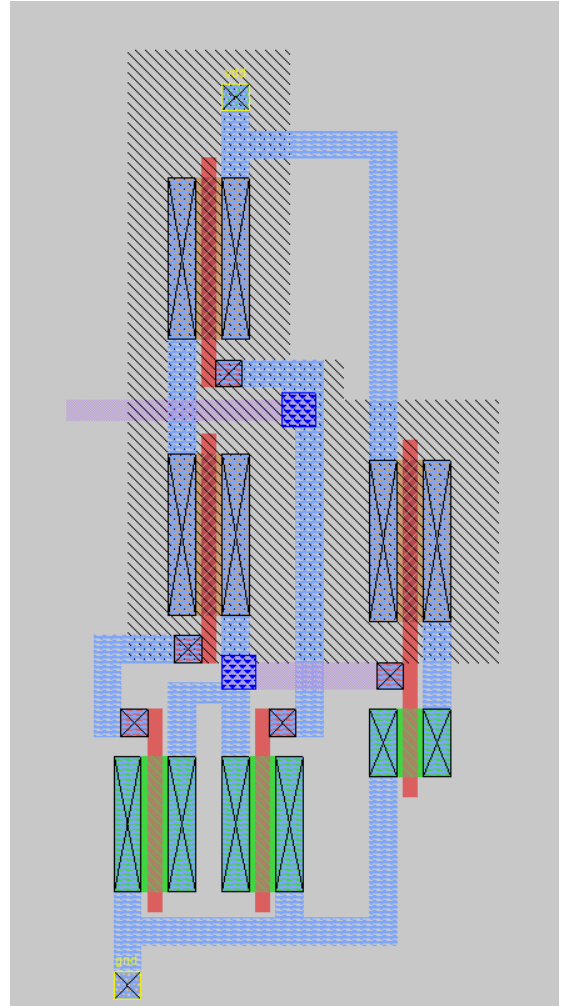


Fig. 6. or magic

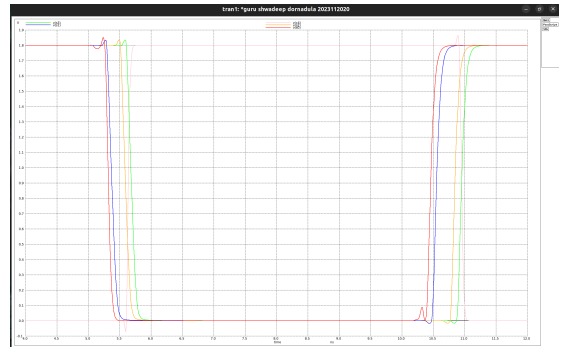


Fig. 7. ngspice cla output

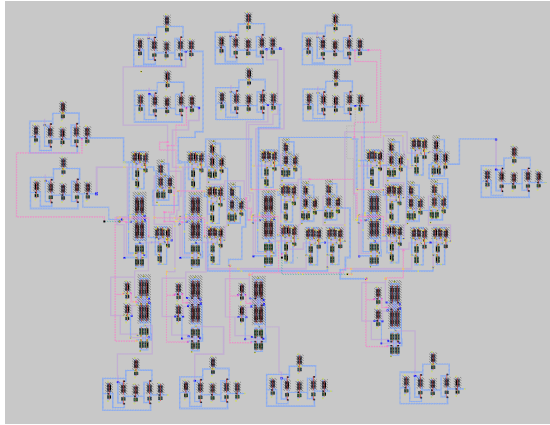


Fig. 8. total magic

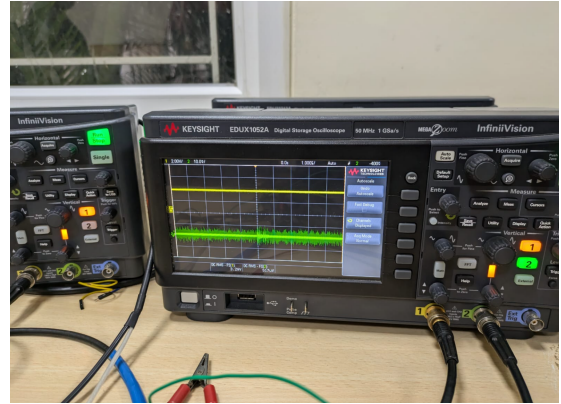


Fig. 11. s3s4

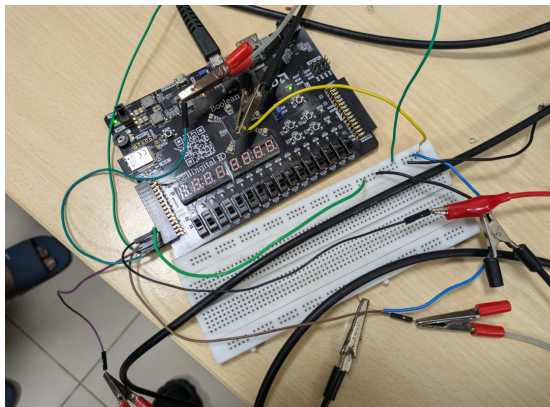


Fig. 9. fpga

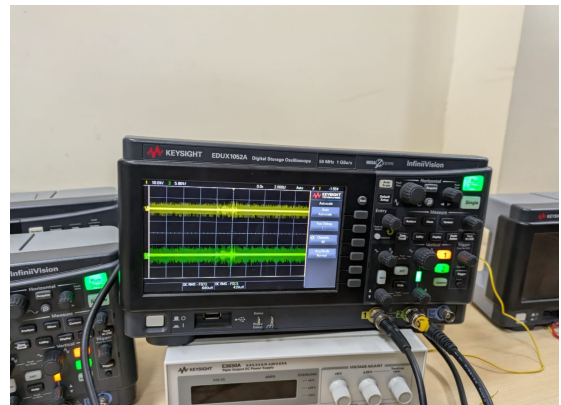


Fig. 12. s1s2



Fig. 10. c4

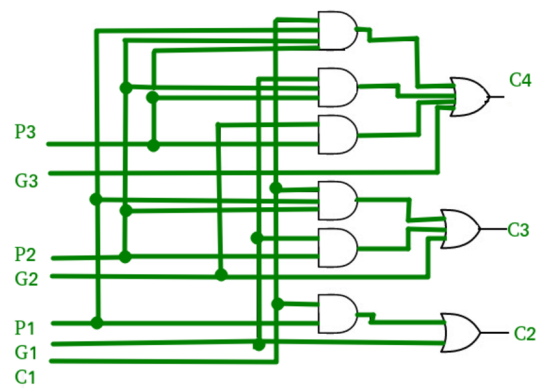


Fig. 13. floorplan

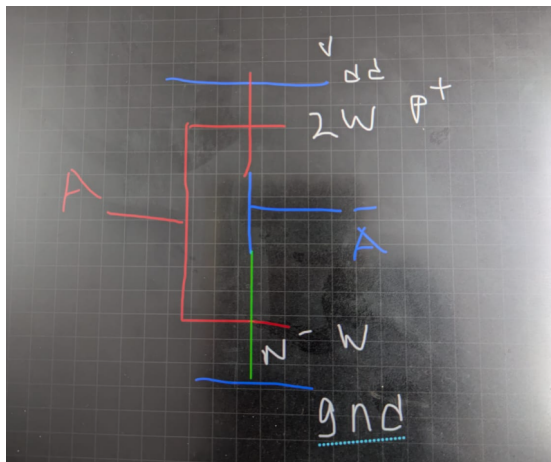


Fig. 14. inverter line diagram

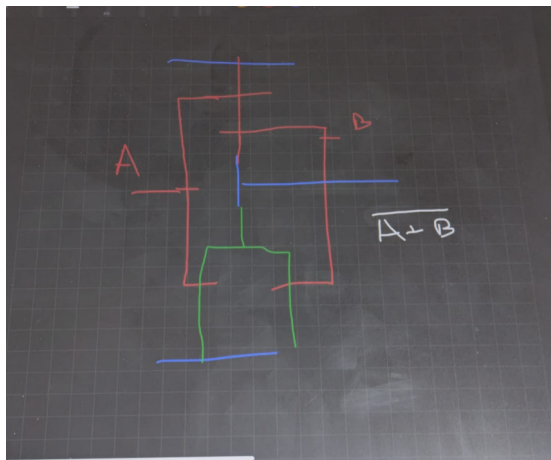


Fig. 15. nand line diagram

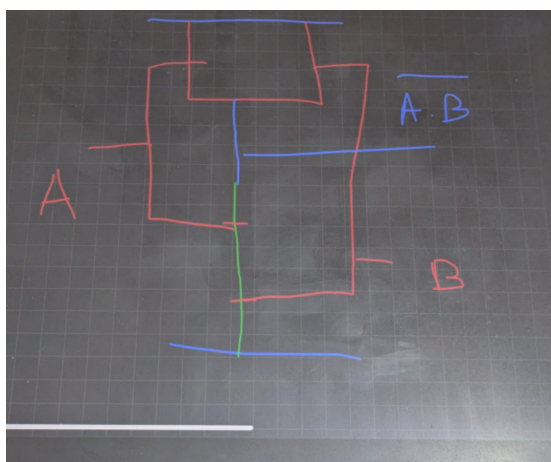


Fig. 16. nor line diagram